

Professional Python Project Companion

An Opinionated Guide to Professional Python Project Infrastructure

Denise M. Case

Table of contents

1 Professional Python Project Companion	4
2 Preface	5
I The Professional Python Project	6
3 Project Infrastructure	7
4 Using the Companion	8
II Project Identity and Structure	9
5 README.md	10
6 LICENSE	11
7 CITATION.cff	12
8 CHANGELOG.md	13
9 src/ Layout	14
III Reproducible Project Configuration	15
10 pyproject.toml	16
11 Python Version	17
12 Lock File	18
IV Consistent Working Environment	19
13 .editorconfig	20
13.1 Purpose	20

13.2 Maintenance	20
13.3 Canonical example	20
14 .gitignore	21
15 .gitattributes	22
16 VS Code Settings	23
17 VS Code Extensions	24
 V Local Quality and Automation	 25
18 Ruff	26
19 Pyright	27
20 pytest	28
21 pre-commit	29
22 Markdownlint	30
23 YAML Linting	31
 VI Shared Automation and Maintenance	 32
24 Continuous Integration	33
25 Documentation Deployment	34
26 Dependabot	35
 VII Project Documentation	 36
 VIII Working with Agentic AI	 38
 IX Putting It All Together	 42
31 Putting It Together	43

1 Professional Python Project Companion

An Opinionated Guide to Professional Python Project Infrastructure

A professional Python project includes more than good source code. It includes the files, conventions, automation, documentation, and validation that allow the project to be developed, maintained, and shared reliably.

This companion explains those parts.

It presents one coherent and intentionally opinionated approach to professional Python project infrastructure. The particular tools and configurations are chosen for their popularity and longevity, but like everything, can be expected to evolve. The engineering concerns of consistency, reproducibility, validation, collaboration, documentation, and maintenance that they address will remain.

Maintained canonical examples are included and may be forked, adapted, and evolved for other projects, teams, and organizations.

2 Preface

This is a concise guide to selected infrastructure for a professional Python project.

The companion is intentionally opinionated. Professional projects require concrete engineering decisions. Examples reflect choices I find valuable enough to use across professionally maintained Python repositories.

Like everything in our rapidly-changing field, choices will evolve.

Readers are encouraged to understand the reasoning, adopt what serves their work, and create their own project standards.

Part I

The Professional Python Project

3 Project Infrastructure

4 Using the Companion

Part II

Project Identity and Structure

5 README.md

6 LICENSE

7 CITATION.cff

8 CHANGELOG.md

9 src/ Layout

Part III

Reproducible Project Configuration

10 pyproject.toml

11 Python Version

12 Lock File

Part IV

Consistent Working Environment

13 .editorconfig

13.1 Purpose

`.editorconfig` defines basic text-file conventions that compatible editors apply automatically.

It helps contributors using different editors and operating systems produce files with consistent indentation, encoding, line endings, and final-newline behavior.

13.2 Maintenance

Change `.editorconfig` when the project adopts different text conventions or begins using file types that require different settings.

Most projects rarely need to modify it after the initial decisions have been made.

13.3 Canonical example

See the maintained annotated [.editorconfig](#).

Workshop companion

`.editorconfig` establishes consistent working conditions. It is the level workbench that allows different people to bring different tools without producing incompatible work.

14 .gitignore

15 .gitattributes

16 VS Code Settings

17 VS Code Extensions

Part V

Local Quality and Automation

18 Ruff

19 Pyright

20 pytest

21 pre-commit

22 Markdownlint

23 YAML Linting

Part VI

Shared Automation and Maintenance

24 Continuous Integration

25 Documentation Deployment

26 Dependabot

Part VII

Project Documentation

Part VIII

Working with Agentic AI

28

29

30

Part IX

Putting It All Together

31 Putting It Together